🚀 Lecture 04.03: String Formatting & Interpolation Systems

📌 Executive Architectural Summary

String interpolation injects dynamic runtime values into structured textual templates. Python evolved from C-style `%` printf formatting to `str.format()` (PEP 3101), culminating in modern **Formatted String Literals (f-strings, PEP 498)**, which evaluate embedded expressions at runtime directly via bytecode instructions (`FORMAT_VALUE`).

🔬 Core Engineering Mechanics

1. Comparative Syntax Evolution

1. **C-Style `%` Operator:** `"Value: %.2f" % val` (Legacy, type-specific).
2. **`str.format()` Method:** `"Value: {:.2f}".format(val)` (Versatile, slower).
3. **F-Strings:** `f"Value: {val:.2f}"` (Direct AST parsing, fastest execution).

2. Format Specifiers & Alignment

- Alignment: `<` (Left-align), `>` (Right-align), `^` (Center-align).
- Float Precision: `.{precision}f` (e.g., `:.4f` for 4 decimal places).
- Comma Separators: `:,` formats 1000000 as `1,000,000` .

💻 Production Python Implementation

```
name = "Shradha"
score = 98.7654
rank = 1

# F-string with formatting specifiers
formatted = f"Student: {name:<10} | Rank: #{rank:02d} | Score: {score:.2f}%"
print(formatted)

# Financial number formatting
```

```
revenue = 14500250.75
print(f"Total Revenue: Rs. {revenue:,.2f}")
```